

Here is a practical, high-impact **Dockerfile Cheatsheet** designed to help you build, optimize, and debug your Docker images quickly.

1. Essential Commands (The Core)

These are the building blocks of almost every Dockerfile.

Instruction	Purpose	Example
FROM	Sets the base image (always start here).	FROM node:20-alpine
WORKDIR	Sets the working directory inside the container.	WORKDIR /app
COPY	Copies files/directories from host to container.	COPY package.json .
RUN	Executes commands during the build phase.	RUN npm install
ENV	Sets persistent environment variables.	ENV PORT=3000
EXPOSE	Documents the port the container listens on.	EXPOSE 3000
CMD	Default command executed when the container starts .	CMD ["npm", "start"]

2. Anatomy of a Standard Dockerfile

Here is how a clean, production-ready Dockerfile for a Node.js application typically looks:

Dockerfile

```
# 1. Use a specific, lightweight base image
FROM node:20-alpine

# 2. Set the working directory
WORKDIR /usr/src/app

# 3. Copy dependency files first (optimizes build caching)
COPY package*.json ./

# 4. Install dependencies
RUN npm ci --only=production

# 5. Copy the rest of the application code
COPY . .

# 6. Set environment variables
ENV NODE_ENV=production
ENV PORT=8080

# 7. Expose the port
EXPOSE 8080

# 8. Define the runtime command
CMD ["node", "server.js"]
```

3. Best Practices & Optimization

Leverage the Build Cache

Docker builds images layer by layer. If a layer hasn't changed, Docker reuses the cached version.

- **Bad:** Copying all files before running install commands. Every minor code change will invalidate the cache and force a slow reinstall of your dependencies.
- **Good:** Copy only package files (package.json, requirements.txt, etc.), run your install command, and *then* copy the rest of your source code.

Use .dockerignore

Keep your images lightweight and secure by preventing unnecessary files from being copied into the build context. Create a .dockerignore file in your root directory:

Plaintext

```
node_modules
.git
.env
Dockerfile
README.md
dist/
```

Minimize Image Size with Multi-Stage Builds

Multi-stage builds allow you to use temporary images to compile or build your app, and then copy only the finished product into a highly stripped-down final image.

Dockerfile

```
# --- Stage 1: Build ---
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# --- Stage 2: Run ---
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
```

```
RUN npm ci --only=production
# Copy only the built assets from the builder stage
COPY --from=builder /app/dist ./dist
```

```
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

4. CMD vs. ENTRYPOINT

These two often cause confusion. Here is the quick breakdown:

- **ENTRYPOINT** sets the command that **will always be executed** when the container starts. It makes your container behave like an executable.
- **CMD** acts as the **default arguments** passed to the ENTRYPOINT. Users can easily override CMD from the command line.

Rule of Thumb: Use ENTRYPOINT for the main command you never want overridden (e.g., python), and CMD for the default file to run (e.g., app.py).

Dockerfile

```
ENTRYPOINT ["python"]
CMD ["app.py"]
```

Running docker run my-image executes: python app.py *Running docker run my-image test.py overrides CMD and executes: python test.py*

5. Security Quick-Wins

- **Don't run as root:** By default, Docker runs container processes as root. Create a non-root user for production.

Dockerfile

```
# Create a system user
```

```
RUN adduser -D appuser
```

```
USER appuser
```

- **Use specific tags:** Avoid FROM ubuntu:latest. If "latest" updates, your build might break. Use specific versions (e.g., FROM ubuntu:22.04).

- **Use Slim/Alpine variants:** Whenever possible, use -slim or -alpine images to reduce your attack surface and download times.

